

claude-windows / da34752f-c664-4262-9602-f5d4cfcc6db4

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\da34752f-c664-4262-9602-f5d4cfcc6db4\subagents\workflows\wf_c5f5bca5-d2a\agent-abff935f228c1d4fe.jsonl`  
- SHA-256: `02b56b044fa29736f9fb56a2904b6452550fa7cc3f4f5f849b1589cd12c802ad`  
- Source modified: `2026-06-12T10:43:40+00:00`  
- Imported at: `2026-07-05T16:48:22+00:00`  
- project: `wf_c5f5bca5-d2a`  
- session_id: `da34752f-c664-4262-9602-f5d4cfcc6db4`
```

Transcript

1. user (2026-06-12T10:37:14.896Z)

Repo: C:\Users\avidu\Projects\sports-data-collector (Windows). Review commit fd530b5 on branch feat/prep-annotation-proxy (diff vs master: run `git diff master...feat/prep-annotation-proxy` in that repo, or read the files directly).

Changed files:

- src/core/proxy.py (NEW - probe/encode/verify/hash/manifest engine)
- src/cli/curate.py (new prep_annotation_proxy method + argparse subparser + dispatch)
- tests/test_proxy.py (NEW - 22 offline tests)
- docs/INGESTION_PIPELINE.md (step 0 + 2 FAQ entries)

Purpose (ADR-002 in docs/DECISIONS.md): re-encode a source video into a downscaled "annotation proxy" for an annotation vendor, while preserving the EXACT frame count / fps / timeline so frame N on the proxy == frame N on the original (the vendor's CVAT labels are recomputed as seconds=frame/fps by src/parsers/cvat/rally_cvat.py). Invariants: never change fps, never trim, verify parity after encode (delete proxy on failure), record provenance (source+proxy MD5) in a manifest CSV. Suite is 159 green; a live ffmpeg-8.1.1 E2E smoke passed.

LENS: integration + contracts with the rest of the repo. Check:

- video_id derivation parity with import-local (curate.py line ~113: `basename.split('.')[0].lower().replace(' ','_')`) - does the new code match EXACTLY (multi-dot filenames!)?
- does anything downstream (convert-cvat, validate-delivery, import-local, export, eval manifest, PIPELINE_ROLE.md contract) break or become misleading because proxies now exist? Is the "share proxy / process original" split stated consistently in code messages + docs?
- gitignore: default out path data/videos/proxies/ - is it really ignored (check .gitignore data/videos/ rule); the manifest default data/annotation_proxy_manifest.csv - tracked intentionally?
- argparse: flag name/dest collisions, defaults sane on Windows paths, help text accuracy vs behavior.
- conventions: lazy imports pattern, print-style UX matching other commands, Python >=3.9 compatibility (typing syntax!), no new hard deps (pyproject), CI (plain pytest) still passes without ffmpeg present on the runner.
- docs: INGESTION_PIPELINE.md step numbering/diagram consistency after adding step 0; FAQ correctness.

2. user (2026-06-12T10:37:20.083Z)

fd530b5 feat(curate): prep-annotation-proxy - frame-identical vendor handoff re-encode (ADR-002)
4fb2d6f docs: ADR-002 - vendor annotation proxies generated in-repo (curate prep-annotation-proxy) (#21)
a2d08f8 docs(cvat): update detect_issues padding rationale to config-driven values (#20)
9e6f204 feat(corpus): ingest the n=6 owned golden corpus (262 rallies, Proprietary/commercial) (#19)
db5e1c2 fix(cvat): fold attribute names so shot_count (singular) maps to shots_count (#18)

```
docs/INGESTION_PIPELINE.md | 29 ++++
src/cli/curate.py          | 147 +++++
src/core/proxy.py          | 242 +++++
tests/test_proxy.py        | 309 +++++
4 files changed, 724 insertions(+), 3 deletions(-)
```

3. user (2026-06-12T10:37:21.464Z)

```
feat/prep-annotation-proxy
M .gitignore
```

4. user (2026-06-12T10:37:26.317Z)

```
diff --git a/docs/INGESTION_PIPELINE.md b/docs/INGESTION_PIPELINE.md
index 3120872..a4f9cf5 100644
--- a/docs/INGESTION_PIPELINE.md
+++ b/docs/INGESTION_PIPELINE.md
@@ -22,9 +22,15 @@ to the annotation spec (`docs/annotation/ROORA_BRIEF.md` +
`ANNOTATION_GUIDE.md`

    slice clips / calibrate / train

...

-## The four commands (in order)
+## The five commands (in order)

    ``bash
+0. BEFORE handoff: re-encode the source into a frame-identical annotation proxy
+# (ADR-002 ? downscaled + scrub-friendly so the vendor can work at frame step 1;
+# same frame count/fps/timeline, parity-verified, provenance recorded).
+# Share the PROXY with the vendor; everything downstream keeps using the ORIGINAL.
+python -m src.cli.curate prep-annotation-proxy --video-path match.mp4
+
+# 1. Normalise a CVAT export ? our canonical per-rally CSV (skip if the vendor sent a CSV)
+python -m src.cli.curate convert-cvat --input job.xml \
+    --video-path match.mp4 --out canonical.csv --issues issues.txt
@@ -118,8 +124,25 @@ python -m backend.tools.rally_slicer --video match.mp4 \
Clips are padded to match the highlight compiler (0.5s lead / 1.0s tail).

#### Does coarse frame sampling hurt boundaries?
-Not in practice: ~0.5s sampling slack is absorbed by the highlight compiler's clip
-padding, so we don't flag sampling density.
+It depends on the consumer. For highlight compilation, ~0.5s sampling slack is
+absorbed by the compiler's clip padding, so validate-delivery doesn't flag sampling
+density. For golden training/eval labels the bar is  $\pm 0.3s$  (Brief §4), and the
+pickleball/table-tennis pilot showed a 30-frame sampling grid at 59.94 fps puts every
+boundary ?0.5s off ? structurally out of spec. That's why videos are handed off as
+annotation proxies (step 0): light enough to annotate at frame step 1, with the
+first/last box of each rally on the exact contact/dead frame.
+
+#### What's an annotation proxy, and why not annotate the original?
+A 4K/59.94 original is too heavy to frame-step in CVAT ? at Phase-B scale (~30-minute
+videos, >100k frames each) it's not realistic at all, which is what pushed the pilot
+vendor onto the coarse sampling grid above. prep-annotation-proxy re-encodes the
+source downscaled (?1080p) with dense keyframes for snappy scrubbing while preserving
+the exact frame count, fps, and timeline (verified after the encode; a failed proxy
+is deleted). Frame N on the proxy IS frame N on the original, so returned labels apply
+to the original unchanged. Provenance (source + proxy MD5) is appended to
+data/annotation_proxy_manifest.csv ? commit it. The proxy is ONLY for annotation:
+import-local, export, and the indexer (which keys videos by file MD5) keep using
+the original. Full rationale: ADR-002 in docs/DECISIONS.md.

#### Will a non-commercial license block export?
The video is kept in the DB regardless of license, but the default export is
diff --git a/src/cli/curate.py b/src/cli/curate.py
index fde4826..a6d76c2 100644
```

```

--- a/src/cli/curate.py
+++ b/src/cli/curate.py
@@ -643,6 +643,120 @@ class SportsDataCLI:
    print(f"Skipped (no annotations) : {skipped_empty}")
    print(f"Manifest          : {manifest_path}\n")

+ def prep_annotation_proxy(self, args):
+     """Re-encode a video into a frame-identical annotation proxy (ADR-002).
+
+     The proxy is what the annotation vendor receives: downscaled and
+     scrub-friendly (dense keyframes) so CVAT frame-step-1 work is cheap,
+     but with the source's EXACT frame count / fps / timeline, so frame N
+     on the proxy is frame N on the original and returned labels apply to
+     the original unchanged. Parity is verified after the encode; a proxy
+     that fails verification is deleted, never shared. Provenance (source +
+     proxy MD5s) is appended to the proxy manifest ? downstream processing
+     (import-local, export, the indexer) keeps running on the ORIGINAL file.
+     """
+     from src.core import proxy as proxylib
+
+     src = args.video_path
+     if not os.path.exists(src):
+         print(f"Error: video '{src}' not found.")
+         sys.exit(1)
+
+     stem = os.path.basename(src).split('.')[0]
+     # Mirror import-local's video_id derivation so the manifest row and a
+     # later DB registration of the same source agree on the key.
+     video_id = args.video_id or stem.lower().replace(" ", "_")
+     out = args.out or os.path.join("data", "videos", "proxies", f"{stem}_proxy.mp4")
+     if os.path.abspath(out) == os.path.abspath(src):
+         print("Error: --out must differ from --video-path.")
+         sys.exit(1)
+     if os.path.exists(out) and not args.force:
+         print(f"Error: proxy '{out}' already exists; pass --force to overwrite.")
+         sys.exit(1)
+
+     try:
+         src_info = proxylib.probe_video(src)
+     except proxylib.ProxyError as e:
+         print(f"Error: {e}")
+         sys.exit(1)
+     if src_info.avg_fps is None:
+         print(f"Error: could not determine fps of '{src}' ? the frame<->time "
+               f"contract needs it.")
+         sys.exit(1)
+     if proxylib.is_vfr(src_info) and not args.allow_vfr:
+         print(f"Error: '{src}' looks variable-frame-rate "
+               f"(r={src_info.r_fps} vs avg={src_info.avg_fps}).")
+         print(" The vendor pipeline recomputes times as frame/fps, which is only "
+               "exact for constant-frame-rate sources. Re-run with --allow-vfr to "
+               "proceed anyway (timestamps stay identical to the original, but "
+               "frame/fps seconds are approximate for BOTH files).")
+         sys.exit(1)
+
+     os.makedirs(os.path.dirname(os.path.abspath(out)), exist_ok=True)
+     cmd = proxylib.build_proxy_command(src, out, height=args.height,
+                                       crf=args.crf, gop=args.gop,
+                                       preset=args.preset)
+     print(f"Encoding proxy ({src_info.resolution} -> height<={args.height}, "
+           f"fps {src_info.avg_fps:g} preserved)...")
+     try:
+         proxylib.run_ffmpeg(cmd)
+         proxy_info = proxylib.probe_video(out)
+     except proxylib.ProxyError as e:

```

```

+         print(f"Error: {e}")
+         sys.exit(1)
+
+ errors = proxylib.verify_parity(src_info, proxy_info)
+ if errors:
+     print("Error: proxy FAILED frame<->time parity verification:")
+     for err in errors:
+         print(f" - {err}")
+     try:
+         os.remove(out)
+         print(f"Deleted the failed proxy '{out}' (a silently-wrong proxy "
+               f"poisons golden labels).")
+     except OSError:
+         print(f"WARNING: could not delete the failed proxy '{out}' ? do NOT "
+               f"share it with the vendor.")
+     sys.exit(1)
+
+ print("Hashing source + proxy (provenance)...")
+ source_md5 = proxylib.md5_file(src)
+ proxy_md5 = proxylib.md5_file(out)
+
+ from datetime import datetime, timezone
+ proxylib.append_manifest_row(args.manifest, {
+     "video_id": video_id,
+     "source_path": os.path.abspath(src),
+     "source_md5": source_md5,
+     "proxy_path": os.path.abspath(out),
+     "proxy_md5": proxy_md5,
+     "fps": f"{src_info.avg_fps:g}",
+     "frame_count": src_info.nb_frames,
+     "duration_s": f"{src_info.duration:.3f}" if src_info.duration is not None else "",
+     "source_resolution": src_info.resolution,
+     "proxy_resolution": proxy_info.resolution,
+     "encoder_settings": (f"libx264 crf={args.crf} g={args.gop} "
+                          f"preset={args.preset} height<={args.height} "
+                          f"fps_mode=passthrough audio=aac160k"),
+     "created_at": datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ"),
+ })
+
+ src_mb = os.path.getsize(src) / (1024 * 1024)
+ out_mb = os.path.getsize(out) / (1024 * 1024)
+ print("\n=== Annotation proxy ready ===")
+ print(f" Video ID      : {video_id}")
+ print(f" Source         : {src} ({src_info.resolution}, {src_mb:.0f} MB, md5
+ {source_md5})")
+ print(f" Proxy          : {out} ({proxy_info.resolution}, {out_mb:.0f} MB, md5
+ {proxy_md5})")
+ print(f" Parity         : VERIFIED ? {src_info.nb_frames} frames @ {src_info.avg_fps:g}
+ fps on both")
+ print(f" Manifest       : {args.manifest}")
+ print("\nNext:")
+ print(" 1. Share the PROXY with the vendor (CVAT at frame step 1; first/last "
+       "box of every rally on the exact contact/dead frame).")
+ print(" 2. On delivery, convert-cvat may use --video-path with either file "
+       "(identical fps/frame count by construction).")
+ print(" 3. import-local / export keep pointing at the ORIGINAL file ? the "
+       "indexer keys videos by its MD5.\n")
+
+ def convert_cvat(self, args):
+     """Convert a vendor CVAT rally export into our canonical per-rally CSV.
+
+ @@ -1017,6 +1131,37 @@ def main():
+     fetch_parser.add_argument("--retry-backoff", type=float, default=20.0, metavar="SECONDS",
+                               help="Base backoff between retries; grows linearly per attempt
+ (default: 20)")

```

```

+ # Prep-annotation-proxy command ? vendor-handoff re-encode (ADR-002)
+ prep_parser = subparsers.add_parser(
+     "prep-annotation-proxy",
+     help="Re-encode a video into a light, frame-identical annotation proxy for "
+         "vendor handoff (ADR-002): downscaled + scrub-friendly, same frame "
+         "count/fps/timeline, parity-verified, provenance recorded")
+ prep_parser.add_argument("--video-path", required=True,
+     help="The ORIGINAL source video (stays the system-of-record file)")
+ prep_parser.add_argument("--out",
+     help="Proxy output path (default: data/videos/proxies/<stem>_proxy.mp4)")
+ prep_parser.add_argument("--video-id",
+     help="Manifest key (default: derived from the filename, like import-local)")
+ prep_parser.add_argument("--height", type=int, default=1080,
+     help="Max proxy height; never upscales (default: 1080 ? "
+         "720 risks losing the table-tennis ball)")
+ prep_parser.add_argument("--crf", type=int, default=19,
+     help="x264 quality, lower=better (default: 19)")
+ prep_parser.add_argument("--gop", type=int, default=15,
+     help="Max keyframe interval in frames; small = snappy "
+         "frame-stepping (default: 15)")
+ prep_parser.add_argument("--preset", default="medium",
+     help="x264 speed/size preset (default: medium)")
+ prep_parser.add_argument("--manifest", default=os.path.join("data",
+ "annotation_proxy_manifest.csv"),
+     help="Provenance CSV to append to (default: data/annotation_proxy_manifest.csv; "
+         "commit it ? it is the proxy<->original mapping of record)")
+ prep_parser.add_argument("--allow-vfr", action="store_true",
+     help="Proceed on a variable-frame-rate source (frame/fps "
+         "seconds are approximate; default: refuse)")
+ prep_parser.add_argument("--force", action="store_true",
+     help="Overwrite an existing proxy file")
+
+ # Convert-cvat command ? normalize a vendor CVAT export into our canonical CSV
+ cvat_parser = subparsers.add_parser(
+     "convert-cvat",
@@ -1077,6 +1222,8 @@ def main():
+     cli.curate()
+     elif args.command == "import-local":
+         cli.import_local(args)
+     elif args.command == "prep-annotation-proxy":
+         cli.prep_annotation_proxy(args)
+     elif args.command == "convert-cvat":
+         cli.convert_cvat(args)
+     elif args.command == "validate-delivery":

```

5. user (2026-06-12T10:37:26.730Z)

```

1     """Annotation-proxy generation for vendor handoff (ADR-002).
2
3     Vendors annotate in CVAT, where every rally timestamp is later recomputed from
4     *frame indices* (`seconds = frame / fps` ? see `src/parsers/cvat/rally_cvat.py`).
5     A 4K/59.94 source is too heavy to frame-step, which is what pushed the pilot
6     vendor onto a ~30-frame sampling grid (~0.5 s) and broke the ±0.3 s boundary bar.
7     The fix is to hand the vendor a light annotation proxy of the same timeline:
8     downscaled and scrub-friendly, but identical in frame count, fps, and duration,
9     so frame N on the proxy IS frame N on the original and returned labels apply to
10    the original unchanged.
11
12    The ADR-002 proxy contract enforced here:
13    - video timestamps pass through untouched (`-fps_mode passthrough`, never
14    -r`): same frame count, same fps, no trim, no leading padding;

```

```

15 - the proxy is VERIFIED against the source (frame count / fps / duration) and a
16 failed proxy must be deleted, never shared ? a silently-wrong proxy poisons
17 golden labels;
18 - provenance is recorded (source + proxy MD5s) so the proxy?original mapping
19 survives the handoff (the indexer keys videos by file MD5; downstream
20 processing must keep running on the ORIGINAL file).
21 """
22 import csv
23 import hashlib
24 import os
25 import shutil
26 import subprocess
27 from dataclasses import dataclass
28 from typing import Any, Dict, List, Optional
29
30
31 class ProxyError(RuntimeError):
32     """A proxy step failed in a way that must stop the handoff (never degrade)."""
33
34
35 #: Columns of the provenance manifest (one row appended per generated proxy).
36 MANIFEST_FIELDS = [
37     "video_id", "source_path", "source_md5", "proxy_path", "proxy_md5",
38     "fps", "frame_count", "duration_s", "source_resolution", "proxy_resolution",
39     "encoder_settings", "created_at",
40 ]
41
42 # Encode defaults (see ADR-002). Height 1080 not 720 (table-tennis ball
43 # visibility); GOP 15 = a keyframe every ~0.25 s at 59.94 fps for snappy
44 # back-stepping; CRF 19 keeps the ball/shuttle legible.
45 DEFAULT_HEIGHT = 1080
46 DEFAULT_CRF = 19
47 DEFAULT_GOP = 15
48 DEFAULT_PRESET = "medium"
49
50
51 @dataclass
52 class ProbeInfo:
53     """The frame?time identity of one video stream, as ffprobe reports it."""
54     width: Optional[int] = None
55     height: Optional[int] = None
56     avg_fps: Optional[float] = None # avg_frame_rate (frames / duration)
57     r_fps: Optional[float] = None # r_frame_rate (the container's base rate)
58     nb_frames: Optional[int] = None # demuxed packet count == frame count
59     duration: Optional[float] = None # video stream duration, else container
60
61     @property
62     def resolution(self) -> str:
63         if self.width and self.height:
64             return f"{self.width}x{self.height}"
65         return "unknown"
66
67
68 def parse_rate(value: Optional[str]) -> Optional[float]:
69     """Parse an ffprobe rational rate ('60000/1001', '30/1') to float fps."""
70     if not value:
71         return None
72     try:
73         if "/" in value:
74             num, den = value.split("/", 1)
75             num_f, den_f = float(num), float(den)
76             if den_f == 0:
77                 return None
78             return num_f / den_f
79         return float(value)

```

```

80         except ValueError:
81             return None
82
83
84 def probe_video(path: str) -> ProbeInfo:
85     """Probe the frame?time identity of `path` via ffprobe (loud on failure).
86
87     Uses ``-count_packets`` (demux-only, no decode) so the frame count is exact
88     yet cheap even for a 30-minute file. Unlike the fetch path's best-effort
89     probe, a failure here RAISES: parity verification is the whole point of the
90     proxy flow, so a missing ffprobe must stop it, not degrade it.
91     """
92     if not shutil.which("ffprobe"):
93         raise ProxyError("ffprobe not found on PATH (install FFmpeg)")
94     if not os.path.exists(path):
95         raise ProxyError(f"video not found: {path}")
96     import json as _json
97     try:
98         out = subprocess.run(
99             ["ffprobe", "-v", "error", "-select_streams", "v:0",
100              "-count_packets",
101              "-show_entries",
102              "stream=width,height,avg_frame_rate,r_frame_rate,nb_read_packets,duration"
103              ":format=duration",
104              "-of", "json", path],
105             stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True, check=True,
106             ).stdout
107         doc = _json.loads(out)
108     except (subprocess.SubprocessError, OSError, ValueError) as e:
109         raise ProxyError(f"ffprobe failed on {path}: {e}")
110
111     streams = doc.get("streams") or []
112     if not streams:
113         raise ProxyError(f"no video stream found in {path}")
114     s = streams[0]
115
116     def _f(value) -> Optional[float]:
117         try:
118             return float(value)
119         except (TypeError, ValueError):
120             return None
121
122     def _i(value) -> Optional[int]:
123         try:
124             return int(value)
125         except (TypeError, ValueError):
126             return None
127
128     # Prefer the video stream's own duration; fall back to the container's
129     # (audio encoder priming can stretch the container a few hundredths).
130     duration = _f(s.get("duration"))
131     if duration is None:
132         duration = _f((doc.get("format") or {}).get("duration"))
133
134     return ProbeInfo(
135         width=_i(s.get("width")),
136         height=_i(s.get("height")),
137         avg_fps=parse_rate(s.get("avg_frame_rate")),
138         r_fps=parse_rate(s.get("r_frame_rate")),
139         nb_frames=_i(s.get("nb_read_packets")),
140         duration=duration,
141     )
142
143
144 def is_vfr(info: ProbeInfo, rel_tol: float = 0.001) -> bool:

```

```

145     """True when the stream looks variable-frame-rate (or we cannot tell).
146
147     For CFR camera footage r_frame_rate == avg_frame_rate. A mismatch means the
148     `frame / fps` time recompute is approximate ? conservative callers should
149     require an explicit opt-in before proxying such a source.
150     """
151     if info.avg_fps is None or info.r_fps is None or info.avg_fps == 0:
152         return True
153     return abs(info.r_fps - info.avg_fps) / info.avg_fps > rel_tol
154
155
156 def build_proxy_command(src: str, dst: str, height: int = DEFAULT_HEIGHT,
157                        crf: int = DEFAULT_CRF, gop: int = DEFAULT_GOP,
158                        preset: str = DEFAULT_PRESET) -> List[str]:
159     """The ffmpeg invocation that downscales WITHOUT touching the timeline.
160
161     Invariants (ADR-002): no ``-r`` / fps filter (fps must never change), no
162     trim, ``-fps_mode passthrough`` (ffmpeg >= 5.1) so every source frame maps
163     1:1 onto a proxy frame with its original timestamp. ``min(height, ih)``
164     never upscales. Audio is kept (contact sound is the annotator's best
165     boundary cue).
166     """
167     return [
168         "ffmpeg", "-hide_banner", "-loglevel", "warning", "-stats", "-y",
169         "-i", src,
170         "-map", "0:v:0", "-map", "0:a?",
171         "-vf", f"scale=-2:min({height}\\,ih)",
172         "-c:v", "libx264", "-preset", preset, "-crf", str(crf),
173         "-g", str(gop), "-pix_fmt", "yuv420p",
174         "-fps_mode", "passthrough",
175         "-c:a", "aac", "-b:a", "160k",
176         "-movflags", "+faststart",
177         dst,
178     ]
179
180
181 def run_ffmpeg(cmd: List[str]) -> None:
182     """Run an ffmpeg command, streaming its progress to the console."""
183     if not shutil.which("ffmpeg"):
184         raise ProxyError("ffmpeg not found on PATH (install FFmpeg)")
185     try:
186         # No capture: -stats progress and any warnings go straight to the user.
187         subprocess.run(cmd, check=True)
188     except (subprocess.SubprocessError, OSError) as e:
189         code = getattr(e, "returncode", "?")
190         raise ProxyError(f"ffmpeg failed (exit {code}) ? proxy not produced")
191
192
193 def verify_parity(src: ProbeInfo, proxy: ProbeInfo) -> List[str]:
194     """Check the proxy carries the source's exact frame?time identity.
195
196     Returns a list of human-readable failures (empty == parity holds). Frame
197     count must match EXACTLY; fps within 0.1%; duration within a small slack
198     (AAC priming can pad the container by a few hundredths of a second) that
199     still catches any real trim.
200     """
201     errors = []
202
203     if src.nb_frames is None or proxy.nb_frames is None:
204         errors.append("frame count unavailable on source and/or proxy ? cannot prove
205 parity")
206     elif src.nb_frames != proxy.nb_frames:
207         errors.append(f"frame count changed: source {src.nb_frames} -> proxy
208 {proxy.nb_frames}")

```



```

208         if src.avg_fps is None or proxy.avg_fps is None:
209             errors.append("fps unavailable on source and/or proxy ? cannot prove parity")
210         elif abs(proxy.avg_fps - src.avg_fps) / src.avg_fps > 0.001:
211             errors.append(f"fps changed: source {src.avg_fps:.3f} -> proxy
{proxy.avg_fps:.3f}")
212
213         if src.duration is None or proxy.duration is None:
214             errors.append("duration unavailable on source and/or proxy ? cannot prove
parity")
215         else:
216             tol = max(2.0 / src.avg_fps, 0.2) if src.avg_fps else 0.2
217             if abs(proxy.duration - src.duration) > tol:
218                 errors.append(f"duration changed: source {src.duration:.3f}s -> "
219                               f"proxy {proxy.duration:.3f}s (tolerance {tol:.3f}s)")
220
221         return errors
222
223
224     def md5_file(path: str, chunk_size: int = 1024 * 1024) -> str:
225         """MD5 of a file's bytes (the identity downstream tooling keys videos by)."""
226         digest = hashlib.md5()
227         with open(path, "rb") as f:
228             for chunk in iter(lambda: f.read(chunk_size), b''):
229                 digest.update(chunk)
230         return digest.hexdigest()
231
232
233     def append_manifest_row(manifest_path: str, row: Dict[str, Any]) -> None:
234         """Append one provenance row, writing the header only on first creation."""
235         parent = os.path.dirname(os.path.abspath(manifest_path))
236         os.makedirs(parent, exist_ok=True)
237         is_new = not os.path.exists(manifest_path) or os.path.getsize(manifest_path) == 0
238         with open(manifest_path, "a", newline="", encoding="utf-8") as f:
239             writer = csv.DictWriter(f, fieldnames=MANIFEST_FIELDS)
240             if is_new:
241                 writer.writeheader()
242             writer.writerow({k: row.get(k, "") for k in MANIFEST_FIELDS})
243

```

6. user (2026-06-12T10:37:34.575Z)

Exit code 128

fatal: ambiguous argument 'feat\prep-annotation-proxy;.gitignore': unknown revision or path not in the working tree.

Use '--' to separate paths from revisions, like this:

'git <command> [<revision>...] -- [<file>...']

7. user (2026-06-12T10:37:35.820Z)

```

63:     def _guard_import(self, sport_str, video_id, annotations, force=False):
64:         # annotations (filename-derived video_ids collide easily).
65:         if self.db.get_video(video_id):
66:             n = len(self.db.get_segment_intervals(video_id, SportType(sport_str)))
67:             print(f"Error: video_id '{video_id}' already has {n} annotated segment(s); "
68:                   f"Warning: --force given ? replacing {n} existing segment(s) for
video_id '{video_id}'.")
69:             match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
70:             video_id = args.video_id or match_name.lower().replace(" ", "_")
71:             annotations = self._parse_local_annotations(sport_str, video_id,
annotations_path)
72:             self._guard_import(sport_str, video_id, annotations, force=getattr(args, "force",
False))
73:             video_id=video_id,
74:             print(f" - Video ID: {video_id}")
75:     def _parse_local_annotations(self, sport: str, video_id: str, filepath: str) ->

```

```

List[Any]:
234:             video_id=video_id,
246:             video_id=video_id,
257:             video_id=video_id,
270:             video_id=video_id,
282:             video_id=video_id,
301:             video_id=video_id,
313:             video_id=video_id,
324:             video_id=video_id,
337:             video_id=video_id,
349:             video_id=video_id,
436:         if args.video_id:
437:             videos = [v for v in videos if v.video_id == args.video_id]
443:         results = [] # (video_id, action, detail)
446:             results.append((video.video_id, "skip", "non-commercial"))
451:             results.append((video.video_id, "skip", "no real source URL"))
454:             dest_path = os.path.join(local_dir, f"{video.video_id}.mp4")
460:             results.append((video.video_id, "have", f"{height_to_label(height)}
already"))
463:             print(f"Fetching {video.video_id} ({video.match_name})...")
467:             results.append((video.video_id, "FAIL", fail_reason))
483:             results.append((video.video_id, "fetched", detail))
520:             if video.video_id in seen:
522:                 seen.add(video.video_id)
535:             print(f"  {v.video_id:40s} {v.license_type.value} (video) -> annotation_license;
"

551:         Emits, per qualifying video, a `<out_dir>/<sport>/<video_id>.csv` in the
555:         indexer keys videos by `file MD5`, not by our `video_id`, so it must run
590:         intervals = self.db.get_segment_intervals(video.video_id, sport)
597:         csv_path = os.path.join(sport_dir, f"{video.video_id}.csv")
605:         "video_id": video.video_id,
633:         summary = [[m["sport"], m["video_id"], m["num_segments"],
665:         stem = os.path.basename(src).split('.')[0]
666:         # Mirror import-local's video_id derivation so the manifest row and a
668:         video_id = args.video_id or stem.lower().replace(" ", "_")
728:         "video_id": video_id,
747:         print(f"  Video ID      : {video_id}")
828:         vid = args.video_id or "delivery"
903:         video.video_id,
934:         rallies = self.db.get_badminton_rallies(video.video_id)
936:         rallies = self.db.get_pickleball_rallies(video.video_id)
938:         rallies = self.db.get_tabletennis_rallies(video.video_id)
940:         pts = self.db.get_tennis_points(video.video_id)
942:         delivs = self.db.get_cricket_deliveries(video.video_id)
947:         print(f"Video ID      : {video.video_id}")
981:         self.db.update_status(video.video_id, CurationStatus.CURATED)
985:         self.db.delete_video(video.video_id)

```

8. user (2026-06-12T10:37:48.151Z)

---- Python ----

```

__pycache__/
*.py[cod]
*$py.class
*.egg-info/
.eggs/
build/
dist/
.installed.cfg
*.egg

```

---- Test / coverage ----

```

.pytest_cache/

```

```
.coverage
.coverage.*
htmlcov/
.tox/
.cache/
```

---- Virtual environments ----

```
.venv/
venv/
env/
ENV/
```

---- Secrets / local config ----

```
.env
.env.*
```

---- OS / editor cruft ----

```
.DS_Store
Thumbs.db
.idea/
.vscode/
*.swp
```

---- Project: large media kept outside git (see README) ----

The video corpus is hundreds of MB+; only metadata (the SQLite DB) and the small source annotation CSVs are version-controlled. Re-fetch videos from the stored video_url / local_path per the manifest.

```
data/videos/
```

---- Generated eval/training exports (regenerated via `curate export`) ----

```
data/eval_export*/
```

---- Generated temp artifacts (crawler scratch space; regenerated each run) ----

```
data/temp/
```

---- Per-video observability reports (written next to videos / inputs at runtime) ----

```
*.report.json
*.report.md
*.summary.md
```

Local DB backups (e.g. data/sports_metadata.backup-YYYY-MM-DD.db) ? never commit

```
data/*.backup-*.db
```

Model artifact bundles: weights are NEVER committed (private Releases); the reviewable surface is each bundle's manifest.json (ADR-008).

```
artifacts/
```

```
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it
```

```
diff --git a/.gitignore b/.gitignore
```

```
index 52dac92..ac1420f 100644
```

```
--- a/.gitignore
```

```
+++ b/.gitignore
```

```
@@ -53,3 +53,7 @@ data/temp/
```

```
# Local DB backups (e.g. data/sports_metadata.backup-YYYY-MM-DD.db) ? never commit
```

```

data/*.backup-*.db
+
+# Model artifact bundles: weights are NEVER committed (private Releases); the
+# reviewable surface is each bundle's manifest.json (ADR-008).
+artifacts/

```

9. user (2026-06-12T10:37:50.023Z)

```

    if skipped_noncomm:
        print(f"Skipped (non-commercial) : {skipped_noncomm} (pass --include-noncommercial
to include)")
    if skipped_empty:
        print(f"Skipped (no annotations) : {skipped_empty}")
    print(f"Manifest      : {manifest_path}\n")

def prep_annotation_proxy(self, args):
    """Re-encode a video into a frame-identical annotation proxy (ADR-002).

    The proxy is what the annotation vendor receives: downscaled and
    scrub-friendly (dense keyframes) so CVAT frame-step-1 work is cheap,
    but with the source's EXACT frame count / fps / timeline, so frame N
    on the proxy is frame N on the original and returned labels apply to
    the original unchanged. Parity is verified after the encode; a proxy
    that fails verification is deleted, never shared. Provenance (source +
    proxy MD5s) is appended to the proxy manifest ? downstream processing
    (import-local, export, the indexer) keeps running on the ORIGINAL file.
    """
    from src.core import proxy as proxylib

    src = args.video_path
    if not os.path.exists(src):
        print(f"Error: video '{src}' not found.")
        sys.exit(1)

    stem = os.path.basename(src).split('.')[0]
    # Mirror import-local's video_id derivation so the manifest row and a
    # later DB registration of the same source agree on the key.
    video_id = args.video_id or stem.lower().replace(" ", "_")
    out = args.out or os.path.join("data", "videos", "proxies", f"{stem}_proxy.mp4")
    if os.path.abspath(out) == os.path.abspath(src):
        print("Error: --out must differ from --video-path.")
        sys.exit(1)
    if os.path.exists(out) and not args.force:
        print(f"Error: proxy '{out}' already exists; pass --force to overwrite.")
        sys.exit(1)

    try:
        src_info = proxylib.probe_video(src)
    except proxylib.ProxyError as e:
        print(f"Error: {e}")
        sys.exit(1)
    if src_info.avg_fps is None:
        print(f"Error: could not determine fps of '{src}' ? the frame<->time "
              f"contract needs it.")
        sys.exit(1)
    if proxylib.is_vfr(src_info) and not args.allow_vfr:
        print(f"Error: '{src}' looks variable-frame-rate "
              f"r={src_info.r_fps} vs avg={src_info.avg_fps}.")
        print(" The vendor pipeline recomputes times as frame/fps, which is only "
              "exact for constant-frame-rate sources. Re-run with --allow-vfr to "
              "proceed anyway (timestamps stay identical to the original, but "
              "frame/fps seconds are approximate for BOTH files).")
        sys.exit(1)

    os.makedirs(os.path.dirname(os.path.abspath(out)), exist_ok=True)

```

```

cmd = proxylib.build_proxy_command(src, out, height=args.height,
                                   crf=args.crf, gop=args.gop,
                                   preset=args.preset)
print(f"Encoding proxy ({src_info.resolution} -> height<={args.height}, "
      f"fps {src_info.avg_fps:g} preserved)...")

```

10. user (2026-06-12T10:37:58.874Z)

```

print(" 3. import-local / export keep pointing at the ORIGINAL file ? the "
      "indexer keys videos by its MD5.\n")

def convert_cvat(self, args):
    """Convert a vendor CVAT rally export into our canonical per-rally CSV.

    Absorbs the CVAT bounding-box delivery format (rally fields carried as box
    attributes) on the ingestion side: each rally's time is recomputed from
    its frame range at the video's true fps, and content defects (overlaps,
    zero-duration, off-vocabulary endings, coarse sampling) are written to an
    issues report rather than silently "fixed". The emitted CSV imports via
    `import-local`; any blocker issue must be resolved first, since the
    importer's validator rejects overlapping / zero-duration rallies.
    """
    from src.parsers.cvat.rally_cvat import (
        convert, write_canonical_csv, render_issue_report,
    )
    try:
        result = convert(args.input, fps=args.fps,
                        video_path=args.video_path, step=args.step)
    except (FileNotFoundError, ValueError) as e:
        print(f"Error: {e}")
        sys.exit(1)

    if not result.records:
        print(f"Error: the CVAT export parsed to ZERO rallies ({args.input}). "
              f"Likely the wrong job was exported or the rally_number attribute was "
              f"renamed (boxes without a matched rally_number are skipped).")
        sys.exit(1)

    write_canonical_csv(result.records, args.out)

    # Observability report next to the canonical CSV. Never raises.
    from src.reporting import emit_delivery_report
    src_name = os.path.splitext(os.path.basename(args.input))[0]
    dur = (result.meta.get("stop_frame") / result.fps) if (result.meta.get("stop_frame") and
result.fps) else None
    emit_delivery_report(
        out_dir=os.path.dirname(os.path.abspath(args.out)) or ".", source=src_name,
        records=result.records, issues=result.issues, fps=result.fps,
video_path=args.video_path,
        video_duration_s=dur, source_format="cvat", step=result.step,
    )

    report = render_issue_report(result, os.path.basename(args.input))
    if args.issues:
        with open(args.issues, "w", encoding="utf-8") as f:
            f.write(report)
    print(report)
    print(f"Canonical CSV written: {args.out} ({len(result.records)} rallies)")
    if result.n_blockers:
        print(f"\n {result.n_blockers} BLOCKER issue(s) ? import-local will reject "
              f"this file until they are resolved (by the vendor or manual review).")
    else:
        print("\n No blockers. Next:")
        print(f"    python -m src.cli.curate import-local --sport <sport> "
              f"--video-path <video> --annotations-path {args.out} --fps {result.fps:g}")

```

```

def validate_delivery(self, args):
    """Run the full validation suite on a rally delivery and emit a report.

    Accepts a CVAT export (--input + --video-path/--fps) or an already-canonical
    CSV (--csv). Writes, into --out-dir:
        <id>_report.md    human report (scoreboard + issues + rallies to review)
        <id>_report.json  machine summary (for CI / a bulk-ingest loop)
        <id>_review.csv   the annotatable manual-verification sheet
    Exits non-zero if any blocker issue exists, so it can gate a bulk pipeline.
    """
    import json
    from src.parsers.cvat.rally_cvat import (
        convert, load_canonical_csv, detect_issues,
        build_summary, render_validation_md, write_review_csv,
    )
    vid = args.video_id or "delivery"
    fps = None
    step = None
    stop_frame = None
    if args.input:
        try:
            result = convert(args.input, fps=args.fps, video_path=args.video_path)
        except (FileNotFoundError, ValueError) as e:
            print(f"Error: {e}")
            sys.exit(1)
        records, issues, fps = result.records, result.issues, result.fps
        step = result.step
        stop_frame = result.meta.get("stop_frame")
        if not records:
            print(f"Error: the CVAT export parsed to ZERO rallies ({args.input}). "
                  f"The wrong job was exported or the rally_number attribute was renamed. "
                  f"GATE: BLOCKED.")
            sys.exit(2)
    elif args.csv:
        try:
            records = load_canonical_csv(args.csv)
        except FileNotFoundError as e:
            print(f"Error: {e}")
            sys.exit(1)
        if not records:
            print(f"Error: no usable rally rows in {args.csv}")
            sys.exit(1)
        issues = detect_issues(records)
    else:
        print("Error: provide --input <cvat.xml> or --csv <canonical.csv>")
        sys.exit(1)

    os.makedirs(args.out_dir, exist_ok=True)
    summary = build_summary(records, issues, source=vid, fps=fps)
    md = render_validation_md(records, issues, source=vid)
    json_path = os.path.join(args.out_dir, f"{vid}_report.json")
    md_path = os.path.join(args.out_dir, f"{vid}_report.md")
    review_path = os.path.join(args.out_dir, f"{vid}_review.csv")
    with open(json_path, "w", encoding="utf-8") as f:
        json.dump(summary, f, indent=2)
    with open(md_path, "w", encoding="utf-8") as f:
        f.write(md)
    write_review_csv(records, issues, review_path)

    # Observability report (envelope JSON + technical md + customer summary),
    # colocated with the existing artifacts. Never raises.
    from src.reporting import emit_delivery_report
    duration_s = (stop_frame / fps) if (stop_frame and fps) else None
    report_paths = emit_delivery_report(
        out_dir=args.out_dir, source=vid, records=records, issues=issues, fps=fps,

```

```

        video_path=args.video_path, video_duration_s=duration_s,
        source_format=("cvat" if args.input else "csv"), step=step,
    )

    print(md)
    print(f"\nWrote:\n {md_path}\n {json_path}\n {review_path}")
    if report_paths:
        print(f" {report_paths.get('technical_md')}\n {report_paths.get('customer_md')}\n
{report_paths.get('json')}")
    if summary["n_blockers"]:
        print(f"\n GATE: BLOCKED ? {summary['n_blockers']} blocker(s) must be resolved.")
        sys.exit(2)
    print("\n GATE: PASS ? review the flagged rallies, then `import-local`.")

def curate(self):
    """Interactive loop to review and curate staging video annotations."""
    staging_videos = self.db.get_videos_by_status(CurationStatus.STAGING)
    if not staging_videos:
        print("\nNo staging video annotations to review.\n")
        return

    print(f"\nFound {len(staging_videos)} matches in STAGING state for curation:")
    table_data = []
    for idx, video in enumerate(staging_videos):

```

11. user (2026-06-12T10:38:07.506Z)

```

1      """Tests for the ADR-002 annotation-proxy flow (src/core/proxy.py + the
2      `prep-annotation-proxy` CLI command).
3
4      Everything runs offline: ffmpeg/ffprobe are mocked; only the pure logic
5      (command construction, parity verification, hashing, manifest append) and the
6      CLI orchestration (refusals, deletion-on-failure, provenance row) are exercised.
7      """
8      import csv
9      import os
10     from types import SimpleNamespace
11     from unittest import mock
12
13     import pytest
14     import yaml
15
16     from src.cli.curate import SportsDataCLI
17     from src.core.proxy import (
18         MANIFEST_FIELDS, ProbeInfo, ProxyError, append_manifest_row,
19         build_proxy_command, is_vfr, md5_file, parse_rate, probe_video,
20         verify_parity,
21     )
22
23
24     # ----- #
25     # Pure logic                                     #
26     # ----- #
27
28     def test_build_command_preserves_the_timeline():
29         cmd = build_proxy_command("in.mp4", "out.mp4")
30         # The ADR-002 invariants: no fps change, no trim, timestamps passed through.
31         assert "-r" not in cmd
32         assert "-ss" not in cmd and "-t" not in cmd and "-to" not in cmd
33         assert cmd[cmd.index("-fps_mode") + 1] == "passthrough"
34         # Scrub-friendly + size knobs at their ratified defaults.
35         assert cmd[cmd.index("-g") + 1] == "15"
36         assert cmd[cmd.index("-crf") + 1] == "19"
37         assert cmd[cmd.index("-vf") + 1] == "scale=-2:min(1080\\,ih)"
38         assert cmd[cmd.index("-movflags") + 1] == "+faststart"

```

```

39         # Audio kept (contact sound is the boundary cue), input before output.
40         assert "0:a?" in cmd
41         assert cmd.index("in.mp4") < cmd.index("out.mp4") == len(cmd) - 1
42
43
44     def test_build_command_custom_knobs():
45         cmd = build_proxy_command("a.mp4", "b.mp4", height=720, crf=22, gop=30,
46 preset="fast")
47         assert cmd[cmd.index("-vf") + 1] == "scale=-2:min(720\\,ih)"
48         assert cmd[cmd.index("-crf") + 1] == "22"
49         assert cmd[cmd.index("-g") + 1] == "30"
50         assert cmd[cmd.index("-preset") + 1] == "fast"
51
52     def test_parse_rate():
53         assert parse_rate("30/1") == 30.0
54         assert abs(parse_rate("60000/1001") - 59.94006) < 1e-4
55         assert parse_rate("25") == 25.0
56         assert parse_rate("0/0") is None
57         assert parse_rate("") is None
58         assert parse_rate(None) is None
59         assert parse_rate("garbage") is None
60
61
62     def test_is_vfr():
63         cfr = ProbeInfo(avg_fps=59.94006, r_fps=59.94006)
64         assert not is_vfr(cfr)
65         vfr = ProbeInfo(avg_fps=59.2, r_fps=60.0)
66         assert is_vfr(vfr)
67         # Unknown rates are treated as VFR (conservative: force the explicit flag).
68         assert is_vfr(ProbeInfo(avg_fps=None, r_fps=30.0))
69         assert is_vfr(ProbeInfo(avg_fps=30.0, r_fps=None))
70
71
72     def _info(frames=5737, fps=59.94006, dur=95.72, w=3840, h=2160):
73         return ProbeInfo(width=w, height=h, avg_fps=fps, r_fps=fps,
74             nb_frames=frames, duration=dur)
75
76
77     def test_verify_parity_ok_and_tolerates_audio_priming():
78         src = _info()
79         # Same frames/fps; container a hair longer (AAC priming) ? still parity.
80         proxy = _info(dur=95.76, w=1920, h=1080)
81         assert verify_parity(src, proxy) == []
82
83
84     def test_verify_parity_catches_frame_count_change():
85         errors = verify_parity(_info(frames=5737), _info(frames=5730, w=1920, h=1080))
86         assert any("frame count changed" in e for e in errors)
87
88
89     def test_verify_parity_catches_fps_change():
90         errors = verify_parity(_info(fps=59.94006), _info(fps=29.97003, w=1920, h=1080))
91         assert any("fps changed" in e for e in errors)
92
93
94     def test_verify_parity_catches_trim():
95         errors = verify_parity(_info(dur=95.72), _info(dur=94.20, w=1920, h=1080))
96         assert any("duration changed" in e for e in errors)
97
98
99     def test_verify_parity_loud_on_unknown_identity():
100         errors = verify_parity(ProbeInfo(), ProbeInfo())
101         assert len(errors) == 3 # frames, fps, duration all unprovable
102

```



```

103
104 def test_md5_file(tmp_path):
105     p = tmp_path / "f.bin"
106     p.write_bytes(b"hello world")
107     assert md5_file(str(p)) == "5eb63bbbe01eeed093cb22bb8f5acdc3"
108
109
110 def test_append_manifest_header_written_once(tmp_path):
111     manifest = tmp_path / "sub" / "proxies.csv"
112     append_manifest_row(str(manifest), {"video_id": "v1", "source_md5": "a"})
113     append_manifest_row(str(manifest), {"video_id": "v2", "source_md5": "b"})
114     with open(manifest, newline="", encoding="utf-8") as f:
115         rows = list(csv.reader(f))
116         assert rows[0] == MANIFEST_FIELDS
117         assert len(rows) == 3
118         assert rows[1][0] == "v1" and rows[2][0] == "v2"
119
120
121 # ----- #
122 # probe_video (ffprobe mocked) #
123 # ----- #
124
125 _FFPROBE_JSON = """
126 {
127     "streams": [{
128         "width": 3840, "height": 2160,
129         "r_frame_rate": "60000/1001", "avg_frame_rate": "60000/1001",
130         "duration": "95.720000", "nb_read_packets": "5737"
131     }],
132     "format": {"duration": "95.768000"}
133 }
134 """
135
136
137 def test_probe_video_parses_ffprobe_json(tmp_path):
138     p = tmp_path / "v.mp4"
139     p.write_bytes(b"x")
140     with mock.patch("src.core.proxy.shutil.which", return_value="ffprobe"), \
141         mock.patch("src.core.proxy.subprocess.run",
142             return_value=SimpleNamespace(stdout=_FFPROBE_JSON)):
143         info = probe_video(str(p))
144         assert (info.width, info.height) == (3840, 2160)
145         assert info.nb_frames == 5737
146         assert abs(info.avg_fps - 59.94006) < 1e-4
147         assert info.duration == pytest.approx(95.72) # stream duration preferred
148         assert info.resolution == "3840x2160"
149
150
151 def test_probe_video_requires_ffprobe(tmp_path):
152     p = tmp_path / "v.mp4"
153     p.write_bytes(b"x")
154     with mock.patch("src.core.proxy.shutil.which", return_value=None):
155         with pytest.raises(ProxyError, match="ffprobe not found"):
156             probe_video(str(p))
157
158
159 def test_probe_video_missing_file():
160     with mock.patch("src.core.proxy.shutil.which", return_value="ffprobe"):
161         with pytest.raises(ProxyError, match="not found"):
162             probe_video("nope.mp4")
163
164
165 def test_probe_video_no_video_stream(tmp_path):
166     p = tmp_path / "v.mp4"
167     p.write_bytes(b"x")

```

```

168         with mock.patch("src.core.proxy.shutil.which", return_value="ffprobe"), \
169             mock.patch("src.core.proxy.subprocess.run",
170                 return_value=SimpleNamespace(stdout='{ "streams": [] }')):
171             with pytest.raises(ProxyError, match="no video stream"):
172                 probe_video(str(p))
173
174
175 # ----- #
176 # CLI: prep-annotation-proxy #
177 # ----- #
178
179 @pytest.fixture
180 def cli(tmp_path):
181     cfg = {
182         "database_path": str(tmp_path / "test.db"),
183         "supported_sports": ["badminton", "pickleball", "tabletennis"],
184         "commercial_licenses": ["MIT", "Apache-2.0", "BSD", "Proprietary"],
185     }
186     cfg_path = tmp_path / "config.yaml"
187     cfg_path.write_text(yaml.safe_dump(cfg), encoding="utf-8")
188     return SportsDataCLI(config_path=str(cfg_path))
189
190
191 def _prep_args(tmp_path, **kw):
192     base = dict(
193         video_path=str(tmp_path / "Match One.mp4"),
194         out=str(tmp_path / "proxies" / "match_proxy.mp4"),
195         video_id=None, height=1080, crf=19, gop=15, preset="medium",
196         manifest=str(tmp_path / "proxy_manifest.csv"),
197         allow_vfr=False, force=False,
198     )
199     base.update(kw)
200     return SimpleNamespace(**base)
201
202
203 def _cfr(frames=100, w=3840, h=2160):
204     return _info(frames=frames, dur=100 / 59.94006, w=w, h=h)
205
206
207 def test_prep_proxy_happy_path(cli, tmp_path):
208     args = _prep_args(tmp_path)
209     (tmp_path / "Match One.mp4").write_bytes(b"original-bytes")
210
211     def fake_ffmpeg(cmd):
212         with open(cmd[-1], "wb") as f:
213             f.write(b"proxy-bytes")
214
215     probes = [_cfr(), _cfr(w=1920, h=1080)] # source probe, then proxy probe
216     with mock.patch("src.core.proxy.probe_video", side_effect=probes), \
217         mock.patch("src.core.proxy.run_ffmpeg", side_effect=fake_ffmpeg):
218         cli.prep_annotation_proxy(args)
219
220     assert os.path.exists(args.out)
221     with open(args.manifest, newline="", encoding="utf-8") as f:
222         rows = list(csv.DictReader(f))
223     assert len(rows) == 1
224     row = rows[0]
225     # video_id derived like import-local: stem lowercased, spaces -> underscores.
226     assert row["video_id"] == "match_one"
227     assert row["source_md5"] == md5_file(str(tmp_path / "Match One.mp4"))
228     assert row["proxy_md5"] == md5_file(args.out)
229     assert row["source_md5"] != row["proxy_md5"]
230     assert row["fps"].startswith("59.94")
231     assert row["frame_count"] == "100"
232     assert row["source_resolution"] == "3840x2160"

```

```

233     assert row["proxy_resolution"] == "1920x1080"
234
235
236 def test_prep_proxy_parity_failure_deletes_proxy(cli, tmp_path):
237     args = _prep_args(tmp_path)
238     (tmp_path / "Match One.mp4").write_bytes(b"original-bytes")
239
240     def fake_ffmpeg(cmd):
241         with open(cmd[-1], "wb") as f:
242             f.write(b"proxy-bytes")
243
244     # Proxy comes back one frame short -> parity check must fail loudly.
245     probes = [_cfr(frames=100), _cfr(frames=99, w=1920, h=1080)]
246     with mock.patch("src.core.proxy.probe_video", side_effect=probes), \
247         mock.patch("src.core.proxy.run_ffmpeg", side_effect=fake_ffmpeg):
248         with pytest.raises(SystemExit) as exc:
249             cli.prep_annotation_proxy(args)
250     assert exc.value.code == 1
251     assert not os.path.exists(args.out), "a failed proxy must never be left on disk"
252     assert not os.path.exists(args.manifest), "no provenance row for a failed proxy"
253
254
255 def test_prep_proxy_refuses_vfr_without_flag(cli, tmp_path):
256     args = _prep_args(tmp_path)
257     (tmp_path / "Match One.mp4").write_bytes(b"original-bytes")
258     vfr_info = ProbeInfo(width=3840, height=2160, avg_fps=59.2, r_fps=60.0,
259                          nb_frames=100, duration=1.69)
260     with mock.patch("src.core.proxy.probe_video", return_value=vfr_info):
261         with pytest.raises(SystemExit) as exc:
262             cli.prep_annotation_proxy(args)
263     assert exc.value.code == 1
264     assert not os.path.exists(args.out)
265
266
267 def test_prep_proxy_allow_vfr_proceeds(cli, tmp_path):
268     args = _prep_args(tmp_path, allow_vfr=True)
269     (tmp_path / "Match One.mp4").write_bytes(b"original-bytes")
270     vfr = ProbeInfo(width=3840, height=2160, avg_fps=59.2, r_fps=60.0,
271                    nb_frames=100, duration=100 / 59.2)
272     vfr_proxy = ProbeInfo(width=1920, height=1080, avg_fps=59.2, r_fps=60.0,
273                          nb_frames=100, duration=100 / 59.2)
274
275     def fake_ffmpeg(cmd):
276         with open(cmd[-1], "wb") as f:
277             f.write(b"proxy-bytes")
278
279     with mock.patch("src.core.proxy.probe_video", side_effect=[vfr, vfr_proxy]), \
280         mock.patch("src.core.proxy.run_ffmpeg", side_effect=fake_ffmpeg):
281         cli.prep_annotation_proxy(args)
282     assert os.path.exists(args.out)
283
284
285 def test_prep_proxy_refuses_existing_output(cli, tmp_path):
286     args = _prep_args(tmp_path)
287     (tmp_path / "Match One.mp4").write_bytes(b"original-bytes")
288     os.makedirs(os.path.dirname(args.out), exist_ok=True)
289     with open(args.out, "wb") as f:
290         f.write(b"existing")
291     with pytest.raises(SystemExit) as exc:
292         cli.prep_annotation_proxy(args)
293     assert exc.value.code == 1
294
295
296 def test_prep_proxy_missing_source(cli, tmp_path):
297     args = _prep_args(tmp_path, video_path=str(tmp_path / "nope.mp4"))

```

```

298     with pytest.raises(SystemExit) as exc:
299         cli.prep_annotation_proxy(args)
300     assert exc.value.code == 1
301
302
303 def test_prep_proxy_out_equals_source_refused(cli, tmp_path):
304     src = tmp_path / "Match One.mp4"
305     src.write_bytes(b"original-bytes")
306     args = _prep_args(tmp_path, out=str(src))
307     with pytest.raises(SystemExit) as exc:
308         cli.prep_annotation_proxy(args)
309     assert exc.value.code == 1
310

```

12. user (2026-06-12T10:38:08.968Z)

```

19: ``seconds = frame / fps`` at the video's true fps. We ignore any
21: samples every ``step`` frames and assumes 30 fps yields systematically wrong
23: sampling step at the wrong fps). The frame index is reliable; the vendor's
47: from src.core.downloader import probe_resolution_fps
109: start_time: float          # seconds, = frame_min / fps
110: end_time: float            # seconds, = frame_max / fps (see padding note)
141: fps: float
246: def build_records(bboxes: List[Dict[str, Any]], fps: float) -> List[RallyRecord]:
248:     if fps is None or fps <= 0:
249:         raise ValueError("fps must be a positive number to convert frames to seconds")
266:         start_time=fmin / fps,
267:         end_time=fmax / fps,
290:     Note: we deliberately do NOT flag the sampling density (step/fps). A boundary
367: def convert(xml_path: str,
368:             fps: Optional[float] = None,
369:             video_path: Optional[str] = None,
372:     """Full pipeline: parse CVAT XML -> records (+ issues) at the true fps.
374:     ``fps`` resolution order: explicit ``fps`` arg > ffprobe of ``video_path`` >
375:     the CVAT meta (none) -> error. We never fall back to a guessed 30 fps, since
380:     if fps is None and video_path:
381:         _, _, probed = probe_resolution_fps(video_path)
382:         fps = probed
383:     if fps is None or fps <= 0:
385:         "Could not determine fps. Pass fps=... or a readable video_path so "
386:         "frames can be converted to seconds (seconds = frame / fps).")
391:     records = build_records(bboxes, fps)
395:     return ConversionResult(records=records, issues=issues, fps=fps, step=step, meta=meta)
421:     f" fps used          : {result.fps:.3f}",
422:     f" sampling step    : {result.step} frames (~{result.step / result.fps:.2f}s)",
506:         source: str = "", fps: Optional[float] = None) -> Dict[str, Any]:
513:     "fps": fps,

```

13. user (2026-06-12T10:39:23.714Z)

```

1     # Rally Ingestion Pipeline ? End-to-End FAQ
2
3     How a vendor's rally annotations become training/eval ground truth, and how we
4     validate each delivery against our conventions. This is the operational companion
5     to the annotation spec (`docs/annotation/ROORA_BRIEF.md` + `ANNOTATION_GUIDE.md`).
6
7     ```
8     vendor delivery          OUR side (collector)          modeling
9     (indexer)
10    ?????????????????? convert ?????????????????? validate ??????????????????
11    ? CVAT export ? ?????????????? ? canonical CSV ? ?????????????? ? report .md ?
12    ? (boxes + attrs)? ? (per rally, ? ? report .json ?
13    ? OR ? ? decimal secs)? ? review .csv ???? human
14    verifies
15    ? canonical CSV ? ?????????????????? ??????????????????

```

```

flagged rallies
14      ??????????????????      ?      ?
15      ? import-local (validated) ?
16      ?      golden labels
17      ?????????????????? export  ??????????????????
18      ? SQLite (rally ? ?????????? ? <id>.csv      ?
start,end (indexer GT)
19      ? tables)      ?      ? manifest.json?
20      ??????????????????      ??????????????????
21      ?
22      slice clips / calibrate /
train
23      ``
24
25      ## The five commands (in order)
26
27      ```bash
28      # 0. BEFORE handoff: re-encode the source into a frame-identical annotation proxy
29      #      (ADR-002 ? downscaled + scrub-friendly so the vendor can work at frame step 1;
30      #      same frame count/fps/timeline, parity-verified, provenance recorded).
31      #      Share the PROXY with the vendor; everything downstream keeps using the ORIGINAL.
32      python -m src.cli.curate prep-annotation-proxy --video-path match.mp4
33
34      # 1. Normalise a CVAT export ? our canonical per-rally CSV (skip if the vendor sent a
CSV)
35      python -m src.cli.curate convert-cvat --input job.xml \
36      --video-path match.mp4 --out canonical.csv --issues issues.txt
37
38      # 2. Validate the delivery ? report + annotatable review sheet (the QA gate)
39      python -m src.cli.curate validate-delivery --input job.xml \
40      --video-path match.mp4 --video-id pilot_badminton_01 --out-dir data/validation
41      #      ...or validate a CSV directly:
42      python -m src.cli.curate validate-delivery --csv canonical.csv --video-id
pilot_badminton_01
43
44      # 3. After human review, import the (corrected) CSV into the DB
45      python -m src.cli.curate import-local --sport badminton \
46      --video-path match.mp4 --annotations-path canonical.csv --fps 59.94
47
48      # 4. Export indexer-ready ground truth (start,end CSVs + manifest)
49      python -m src.cli.curate export --out-dir data/eval_export
50      ```
51
52      ---
53
54      ## FAQ
55
56      ### What delivery formats do we accept?
57      Both. A CVAT export (image- or track-mode XML ? rally fields carried as box
58      attributes) is normalised by `convert-cvat`; an already-canonical CSV is taken
59      as-is. Vendors may keep working in CVAT ? we do not require them to hand-author
60      CSVs.
61
62      ### What is the "canonical" CSV?
63      One row per rally:
64      `rally_number, start_mmss, end_mmss, start_time, end_time, shots_count, ending_reason,
last_shot_outcome, score_before, score_after, notes`.
65      `start_time`/`end_time` are decimal seconds; `import-local` reads those.
66
67      ### How are timestamps handled? (the fps thing)
68      We recompute each rally's interval from its frame range × the true fps
69      (`seconds = frame / fps`). A vendor's CVAT time attributes are ignored ? a
70      step-sampled job that assumes 30 fps writes systematically wrong times. fps comes
71      from `--fps` or fprobe of `--video-path`. `mm:ss` is also accepted everywhere.
72

```

```

73     ### What does validation check?
74     | code | severity | meaning |
75     |---|---|---|
76     | `overlap` | **blocker** | two rallies' intervals overlap (`end[N] > start[N+1]`) |
77     | `zero_duration` | **blocker** | `end <= start` (single-sample / stray box) |
78     | `frame_out_of_range` | high | a box frame is past the video's last frame |
79     | `shot_density` | medium | long rally with implausibly few shots ? likely an **inflated
span** (often hides a merged rally) |
80     | `possible_missed_rally` | medium | a long unlabeled gap between two rallies ? check
for an **omitted rally** |
81     | `ending_reason_vocab` | medium | `ending_reason` outside the agreed set |
82     | `outcome_vocab` | low | `last_shot_outcome` outside `{in,net,out,n_a}` |
83
84     ### Blocker vs. review ? what's the gate?
85     `validate-delivery` exits **non-zero if any blocker exists** (so it can gate a bulk
86     loop / CI). Non-blocker issues don't stop the gate but populate the **review sheet**
87     for the human pass. `import-local` independently rejects overlaps/zero-duration, so
88     bad ground truth can't slip into the DB.
89
90     ### How are *missed rallies* caught? (the worst error)
91     Two complementary checks, because a miss shows up two ways:
92     1. **Clean omission** (a real hole in the timeline) ? `possible_missed_rally` (gap
heuristics).
93     2. **Merged/inflated** (the missed rally got swallowed into a neighbour's box, leaving
94     no gap) ? `shot_density` flags the over-long, under-shot rally, and the human finds
95     the second rally inside it.
96     Neither is a guarantee ? they **surface regions to review**, they don't certify
97     completeness. (A future enhancement: a motion second-opinion over the raw video.)
98
99     ### What's the review sheet?
100     `_review.csv` ? one row per rally with `auto_flags` (which checks fired) plus
101     blank `VERIFIED` / `true_start_mmss` / `true_end_mmss` / `true_ending_reason` /
102     `true_last_shot_outcome` / `reviewer_comment` columns. Open it next to the video
103     (timestamp overlay on) and fill the flagged rows. This is the human gate that turns
104     a delivery into trusted golden labels.
105
106     ### Why two ending fields (`ending_reason` + `last_shot_outcome`)?
107     They're orthogonal: `ending_reason` is *why* the point ended (winner / forced_error /
108     unforced_error / service_fault / let / other); `last_shot_outcome` is *where* the
109     last shot went (in / net / out / n_a). A shuttle that flew long is
110     `forced_error/unforced_error` + `out` ? both kept, no information lost.
111
112     ### How do I onboard a brand-new video?
113     1. Get fps into the intake manifest (we provide it to the vendor).
114     2. `convert-cvat` (if CVAT) ? `validate-delivery` ? read the report, fill the review
115     sheet for flagged rallies ? correct the CSV ? `import-local` ? `export`.
116     3. The same suite runs for every video, so quality stays consistent at scale.
117
118     ### How do I pull a single rally out to eyeball it?
119     The slicer (indexer):
120     ```bash
121     python -m backend.tools.rally_slicer --video match.mp4 \
122     --labels golden_labels.csv --rallies 7,16 --out-dir output/rally_clips
123     ```
124     Clips are padded to match the highlight compiler (0.5s lead / 1.0s tail).
125
126     ### Does coarse frame sampling hurt boundaries?
127     It depends on the consumer. For **highlight compilation**, ~0.5s sampling slack is
128     absorbed by the compiler's clip padding, so `validate-delivery` doesn't flag sampling
129     density. For **golden training/eval labels** the bar is ±0.3s (Brief §4), and the
130     pickleball/table-tennis pilot showed a 30-frame sampling grid at 59.94 fps puts *every*
131     boundary ?0.5s off ? structurally out of spec. That's why videos are handed off as
132     annotation **proxies** (step 0): light enough to annotate at frame step 1, with the
133     first/last box of each rally on the exact contact/dead frame.
134

```

```

135     ### What's an annotation proxy, and why not annotate the original?
136     A 4K/59.94 original is too heavy to frame-step in CVAT ? at Phase-B scale (~30-minute
137     videos, >100k frames each) it's not realistic at all, which is what pushed the pilot
138     vendor onto the coarse sampling grid above. `prep-annotation-proxy` re-encodes the
139     source downsampled (?1080p) with dense keyframes for snappy scrubbing while preserving
140     the **exact frame count, fps, and timeline** (verified after the encode; a failed proxy
141     is deleted). Frame N on the proxy IS frame N on the original, so returned labels apply
142     to the original unchanged. Provenance (source + proxy MD5) is appended to
143     `data/annotation_proxy_manifest.csv` ? commit it. The proxy is ONLY for annotation:
144     `import-local`, `export`, and the indexer (which keys videos by file MD5) keep using
145     the original. Full rationale: ADR-002 in `docs/DECISIONS.md`.
146
147     ### Will a non-commercial license block export?
148     The video is **kept** in the DB regardless of license, but the default `export` is
149     **commercial-only** ? a non-commercial source (e.g. `CC-BY-NC-SA`) is *excluded*
150     from the eval/training set unless you pass `--include-noncommercial`. So if a video
151     "goes missing" downstream, check its `is_commercial` flag first. (The observability
152     report raises a `license_noncommercial` warning for exactly this case.)
153
154     ### What is the observability report? (`*.report.json` / `*.report.md` / `*.summary.md`)
155     Every `validate-delivery`, `convert-cvat`, and `import-local` run now also writes a
156     per-video **observability report** (next to the video for `import-local`; into the
157     `--out-dir` / next to the canonical CSV otherwise):
158     - **`<id>.report.json`** ? machine-readable: every stage, metric, and finding. Diff
159       two runs to see what changed between deliveries.
160     - **`<id>.report.md`** ? the **technical** report: parse / validation / registration
161       stages plus **findings** (one per delivery issue, each citing this FAQ). Read this
162       to debug a delivery.
163     - **`<id>.summary.md`** ? a **delivery summary** for the vendor / data team: the
164       rally count and coverage plus the plain-language list of rallies to review ? no
165       internal codes or severities.
166     This is in addition to the existing `<id>_report.md` / `_report.json` / `_review.csv`
167     (the human review sheet), which are unchanged.
168

```

14. user (2026-06-12T10:39:25.120Z)

```

C:/Users/avidu/Projects/sports-data-collector:
README.md
__pycache__
cleanup.py
config
data
docs
pyproject.toml
src
tests

C:/Users/avidu/Projects/sports-data-collector/.github/workflows:
ci.yml

C:/Users/avidu/Projects/sports-data-collector/docs:
DECISIONS.md
INGESTION_PIPELINE.md
MANUAL_INGESTION_POSITIONING.md
PIPELINE_ROLE.md
annotation

```

15. user (2026-06-12T10:39:34.334Z)

```

1     [build-system]
2     requires = ["setuptools>=61.0.0"]
3     build-backend = "setuptools.build_meta"
4
5     [project]

```

```

6     name = "sports-data-collector"
7     version = "0.1.0"
8     description = "A collector and curator for sports video datasets with sport-specific
schemas and license checks."
9     readme = "README.md"
10    requires-python = ">=3.9"
11    dependencies = [
12        "pydantic>=2.0.0",
13        "pyyaml>=6.0",
14        "tabulate>=0.9.0"
15    ]
16
17    [project.optional-dependencies]
18    # Video acquisition (`fetch` / crawler). Also requires the external binaries
19    # ffmpeg (FFmpeg) and, for YouTube high-res, a JS runtime such as deno ?
20    # neither is pip-installable; see README "External requirements".
21    fetch = [
22        "yt-dlp>=2024.1.0"
23    ]
24    dev = [
25        "pytest>=7.0.0",
26        "pytest-cov>=4.0.0"
27    ]
28    # Per-video observability reports (SOFT dependency: ingestion degrades to a no-op
29    # if absent). Shared, permissively-licensed report toolkit. Pin a tag once the
30    # private repo is published, e.g.:
31    # obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3
32    # For local development install it editable from a sibling checkout:
33    # pip install -e ../sports-obsreport --no-build-isolation
34    report = [
35        "obsreport>=0.1.3",
36    ]
37

```

16. user (2026-06-12T10:39:35.275Z)

```

1     name: CI
2
3     on:
4         push:
5             branches: [master]
6         pull_request:
7
8     jobs:
9         test:
10            runs-on: ubuntu-latest
11            steps:
12            - uses: actions/checkout@v4
13            - uses: actions/setup-python@v5
14              with:
15                python-version: "3.11"
16            - name: Install dependencies
17              run: |
18                python -m pip install --upgrade pip
19                pip install -e ".[dev]"
20            - name: Run tests
21              run: python -m pytest -q
22

```

17. user (2026-06-12T10:39:35.574Z)

```

1     # Role in the rally-segmenter training pipeline
2
3     This repo is the data producer in a two-repo system. Its job is to curate
4     sports videos with per-rally `[start, end)` annotations and commercial-license

```



```

5     metadata, then hand that data to the **badminton-highlight-indexer**, which
6     processes, scores, and (Phase 4) *trains* an offline rally segmenter on it.
7
8     This doc explains *why* the `export` and `fetch` commands exist and how they fit
9     the bigger picture. The full decision records and end-to-end narrative live in the
10    indexer repo:
11
12    - `badminton-highlight-indexer/docs/ROADMAP.md` ? the four-phase plan and end-state.
13    - `badminton-highlight-indexer/docs/DECISIONS.md` ? ADR-002 (this bridge), ADR-003
14      (acquisition policy), ADR-004 (offline learned segmenter).
15
16    ## Why this repo matters
17
18    The indexer's best rally mode refines local detections with a **paid Gemini API**.
19    The strategic goal is to replace that with an **offline model trained on our own
20    commercially-licensed, annotated corpus** ? making rally detection free, offline,
21    and self-improving. None of that is possible without a clean, licensable,
22    accurately-labelled, full-resolution dataset. That dataset is what this repo
23    produces.
24
25    ## What the two key commands do (and why)
26
27    - **`curate export`** ? emits the indexer's ground-truth format: per-video
28      `start,end` CSVs + a `manifest.json` pairing each CSV with its video file.
29      Sport-agnostic (all supported sports ? badminton, tennis, cricket, pickleball,
30      table tennis ? share a `(video_id, ordinal, start, end)` shape) and gated on
31      `is_commercial`, so only licensable data flows
32      downstream. The annotations are **temporal-only** ? they train/evaluate rally
33      *segmentation*, not the frame-level shuttle detector (see ADR-002).
34
35    - **`fetch`** ? (re)downloads videos at **best available resolution**, updating
36      the DB **in place** so annotations and curation status are never lost. Uses
37      authenticated cookies for reliable, unthrottled downloads. The why and the
38      hard-won pitfalls (SABR?144p, Chrome DPAPI, rate-limiting) are in ADR-003.
39
40    ## The one contract that matters: video identity
41
42    The indexer keys videos by **file MD5**, this repo by a human `video_id`. A
43    re-encode/re-download changes the bytes, hence the MD5. So the manifest carries the
44    exact `local_path`, and any downstream processing must run on **that** file. This is
45    why acquisition (`fetch`) happens *before* the indexer processes anything.
46

```

18. user (2026-06-12T10:39:51.671Z)

```

..... [ 45%]
..... [ 90%]
..... [100%]
159 passed in 2.49s

```

19. user (2026-06-12T10:39:51.799Z)

```

112: def probe_resolution_fps(path: str) -> Tuple[Optional[int], Optional[int], Optional[float]]:
113-     """Return (width, height, fps) of a video file via ffprobe, best-effort.
114-
115-     Returns (None, None, None) if ffprobe is unavailable or the file can't be
116-     parsed, so a missing ffprobe degrades gracefully rather than crashing a fetch.
117-     """
118-     if not shutil.which("ffprobe") or not os.path.exists(path):
119-         return (None, None, None)
120-     try:
121-         out = subprocess.run(
122-             ["ffprobe", "-v", "error", "-select_streams", "v:0",
123-              "-show_entries", "stream=width,height,avg_frame_rate",
124-              "-of", "default=noprint_wrappers=1:nokey=1", path],

```

```

125-         stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True, check=True,
126-     ).stdout.split()
127- except (subprocess.SubprocessError, OSError):
128-     return (None, None, None)
129-
130- width = int(out[0]) if len(out) > 0 and out[0].isdigit() else None
131- height = int(out[1]) if len(out) > 1 and out[1].isdigit() else None
132- fps = None
133- if len(out) > 2 and "/" in out[2]:
134-     num, den = out[2].split("/")
135-     try:
136-         den_f = float(den)
137-         fps = round(float(num) / den_f, 3) if den_f else None
138-     except ValueError:
139-         fps = None
140- return (width, height, fps)
141-
142-
143-@dataclass
144-class DownloadResult:
145-    success: bool
146-    path: Optional[str] = None
147-    size_mb: float = 0.0
148-    width: Optional[int] = None
149-    height: Optional[int] = None
150-    fps: Optional[float] = None
151-    resolution_label: Optional[str] = None
152-    error: Optional[str] = None

```

20. user (2026-06-12T10:40:14.671Z)

3:Lightweight ADRs for this repo. (The cross-repo training-pipeline ADRs live in
8:## ADR-001 ? Keep explicit per-sport code instead of a sport registry (for now)
41:- If the cost rises, ADR-001 should be revisited and superseded.
65:## Checklist: adding a new sport today (under ADR-001)
76:## ADR-002 ? Annotation proxies are generated in-repo (`curate prep-annotation-proxy`), not
by a one-off tool
146:- Implementation is a separate, owner-gated PR; this ADR records the agreed choice.

21. user (2026-06-12T10:40:19.681Z)

ADR-002 ? Annotation proxies are generated in-repo (`curate prep-annotation-proxy`), not by a one-off tool

Status: Accepted · **Date:** 2026-06-12 (owner-ratified)

Context

The Roora pilot showed the vendor annotates in CVAT directly on our **4K / 59.94 fps originals** and copes with the decode cost by sampling **every 30th frame**. The result (pickleball + table-tennis review, 2026-06-10): **every rally boundary off by ~0.5 s, up to ~2 s**, against the ± 0.3 s bar (Brief §4) ? all 24 boundaries sit exactly on the 30-frame grid (~0.5 s at 59.94 fps), so they **structurally cannot** meet spec. The same grid was flagged **[HIGH]** in the earlier badminton round (`data/roora\_badminton\_issues.txt`); it is their standing workflow, not a one-off lapse.

Phase B makes this untenable, not just suboptimal: ~12?15 videos **per sport** at ~30 min each (Brief §3). A 30-minute 4K/59.94 file is **>100k frames**; frame-accurate annotation (frame-step 1) on 4K originals at that scale is not realistic for the vendor. A lightweight **annotation proxy** (downscaled, scrub-friendly re-encode of the identical timeline) is therefore close to a **precondition for Phase B**, not an optimization.

The pipeline is proxy-friendly by construction: annotations join by the human-assigned **video_id** (not file MD5), and **convert-cvat** recomputes all times as **frame / fps**, ignoring vendor time attributes. The Brief's "do not re-encode" rule binds the **vendor** (timeline + checksum identity of the file they annotate), not us ? provided downstream

processing always runs on the **original** file (see `PIPELINE\_ROLE.md`: the indexer keys by MD5).

Decision

We pre-process every video before vendor handoff into an annotation proxy, and the generator lives in this repo as a `curate prep-annotation-proxy` subcommand ? not a standalone script, and not in the indexer.

Proxy contract (invariants the tool must enforce)

- **Same fps as the source ? never reduce it.** At a lower fps every CVAT frame index changes meaning, and the vendor's 30-frame sampling habit at 30 fps would become a **1.0 s** grid ? twice as coarse. Same frame count, no trimming, no leading padding ? proxy seconds map 1:1 onto the original.
- **Downscale 4K ? 1080p** (not 720p: table-tennis ball visibility), CRF ~18?20, **dense keyframes** (`-g 15`) so frame-stepping/back-scrubbing is fast, **audio kept** (contact sound is the annotator's best boundary cue).
- **Verification built in:** assert proxy frame count == original frame count and durations match (also surfaces VFR sources, where `frame / fps` is already unsafe).
- **Provenance:** emit/append the intake-manifest row ? `video\_id`, fps, frame count, **original MD5 + proxy MD5** ? so the proxy?original mapping is durable and downstream processing provably targets the original.

Rationale (why the collector, why not a loose script)

- **This repo owns the vendor flow end-to-end** (`docs/annotation/`, intake manifest, `convert-cvat` ? `validate-delivery` ? `import-local`); proxy prep is simply command #0 of that existing pipeline, and it already shells out to ffprobe.
- **The ffmpeg invocation is the trivial 20%.** The durable value is the parity verification + manifest provenance ? exactly what one-off scripts skip. We have precedent for the failure mode: unfilled intake-manifest TODOs, eval artifacts lost in `%TEMP%`, and indexer predictions once keyed under a **downscaled copy's** MD5.
- **Phase-B frequency:** ~40 runs across sessions/operators ? needs tests + CI, i.e. repo-resident code.
- **Rejected ? standalone script:** acceptable stopgap for an immediate redelivery, but no durable verification/provenance. **Rejected ? indexer:** it has the re-encode primitive (`extract\_video\_chunk`) but is the product pipeline; the vendor handoff workflow, manifest, and ingestion all live here.

Consequences / follow-ups

- **A proxy alone does not fix boundaries.** It removes the performance excuse; the binding fix is the convention from the 2026-06-10 review, to be written into the Phase-B brief revision: **annotate at frame step 1; place the first/last box of every rally on the exact contact/dead frame; we supply annotation-optimized proxies.**
- **Re-add a grid-snap QA check to `validate-delivery`.** **`detect\_issues`** deliberately dropped the **`coarse\_sampling`** flag on the grounds that highlight-compiler padding absorbs ~0.5 s slack ? valid for highlight **consumption**, invalid for **±0.3 s** golden **training labels**. A cheap "most boundaries share a common frame stride" flag would have auto-caught this delivery.
- Implementation is a separate, owner-gated PR; this ADR records the agreed choice.

22. user (2026-06-12T10:40:29.063Z)

README.md:126: - [INTAKE_MANIFEST_TEMPLATE.csv](docs/annotation/INTAKE_MANIFEST_TEMPLATE.csv) ? per-video intake sheet.

docs\INGESTION_PIPELINE.md:113:1. Get fps into the intake manifest (we provide it to the vendor).

docs\annotation\R00RA_BRIEF.md:117:- Per-video `fps` + `video\_id`: provided in the intake manifest ? template [INTAKE_MANIFEST_TEMPLATE.csv](./INTAKE_MANIFEST_TEMPLATE.csv).

TODO(avidullu): fill the manifest (one row per pilot video) and share its link.

docs\DECISIONS.md:117:- **Provenance:** emit/append the intake-manifest row ? `video\_id`, fps, frame count,

docs\DECISIONS.md:122:- **This repo owns the vendor flow end-to-end** (`docs/annotation/`, intake manifest,

docs\DECISIONS.md:127: precedent for the failure mode: unfilled intake-manifest TODOs, eval artifacts lost in

23. user (2026-06-12T10:40:34.359Z)

video_id, sport, format, match_name, source_filename, drive_link, fps, duration_mmss, expected_rallies, assigned_to, status, notes
pilot_badminton_01, badminton, singles, TODO(avidullu): match name, TODO(avidullu): exact source filename, TODO(avidullu): paste Drive link to this video, TODO(avidullu): fps e.g. 30, TODO(avidullu): length e.g. 2:45, 10-12,, todo, deliver as pilot_badminton_01.csv
pilot_pickleball_01, pickleball, doubles, TODO(avidullu): match name, TODO(avidullu): exact source filename, TODO(avidullu): paste Drive link to this video, TODO(avidullu): fps e.g. 30, TODO(avidullu): length e.g. 2:30, 10-12,, todo, deliver as pilot_pickleball_01.csv
pilot_tabletennis_01, tabletennis, singles, TODO(avidullu): match name, TODO(avidullu): exact source filename, TODO(avidullu): paste Drive link to this video, TODO(avidullu): fps e.g. 30, TODO(avidullu): length e.g. 2:20, 10-12,, todo, deliver as pilot_tabletennis_01.csv
53: re-encode the video ? our pipeline identifies a video by its file checksum.

24. user (2026-06-12T10:41:22.110Z)

43:Core curation/import/export/status commands work without any of these.
49:All commands should be executed from the root of the project. Make sure `PYTHONPATH` is set to the current directory:

25. user (2026-06-12T10:41:28.199Z)

3:A clean, extensible Python codebase and CLI toolchain designed to discover, ingest, validate, and curate sports video datasets. The repository stores annotations and video metadata in an SQLite database, supports sport-specific tables and schemas, manages video storage outside of Git, and enforces strict commercial license compliance.
5:> **Pipeline role:** this repo is the ***data producer*** feeding the ****badminton-highlight-indexer****, which trains/evaluates an offline rally segmenter on this curated, commercially-licensed data. See docs/PIPELINE_ROLE.md for why `export`/`fetch` exist and how the two repos fit together.
15:- **Local Ingestion Mode:** Manually registers local videos and associated CSV/JSON annotation files directly into the database as curated entries.
27: pip install -e . # core: curate / import-local / export / status
28: pip install -e ".[fetch]" # adds yt-dlp for video acquisition (fetch / crawler)
34:The `fetch` command and the ShuttleSet crawler shell out to external binaries.
40: YouTube's challenge for high-resolution formats. `fetch` auto-detects a deno
43:Core curation/import/export/status commands work without any of these.
57:To check the current statistics of staging vs. curated items:
59:python -m src.cli.curate status
71:python -m src.cli.curate curate
77:python -m src.cli.curate import-local --sport badminton --video-path "data/videos/my_custom_match.mp4" --annotations-path "data/mock_local_badminton_anno.json" --match-name "My Custom Local Match" --license Proprietary
79:`import-local` works for every configured sport ? `badminton`, `tennis`, `cricket`, `pickleball`, and `tabletennis` ? from either a JSON or a CSV annotations file. For the canonical rally CSV schema and the rater-facing instructions used to produce these files (e.g. via an annotation vendor), see docs/annotation/.
82:Videos are ****always downloaded at the best resolution available**** (via yt-dlp `bv\*+ba/b -S res,...`, merged to MP4); the actual width/height/fps is ffprobed and stored ? never assumed. The `fetch` command (re)downloads videos for entries already in the DB and updates `local\_path`/`resolution`/`fps` ****in place****, preserving all rally annotations and curation status (no clean slate needed):
85:python -m src.cli.curate fetch --cookies-from-browser firefox
88:python -m src.cli.curate fetch --video-id shuttleset_1 --force --cookies-from-browser firefox
90:Flags: `--sport`, `--video-id`, `--status {staging,curated,all}` (default `all`), `--min-height` (skip videos already at/above it unless `--force`; default `720`), `--max-height` (cap resolution, e.g. `1080`; default best available), `--force`, `--include-noncommercial`, `--cookies-from-browser`/`--cookies`, `--player-client`. A global resolution cap can also be set via `video\_storage.max\_height` in `config/config.yaml`. Re-runs are cheap and resumable; a download that falls below `--min-height` is treated as a ****failure**** (discarded, DB left unchanged) so a flaky low-res result never overwrites a good entry.
94:> **JS runtime:** yt-dlp needs one (e.g. [deno](https://deno.com)) to solve YouTube's challenges for high-res formats; `fetch` auto-detects a deno install at `~/deno/bin`. The default `--player-client` avoids `web`/`web\_safari`, which YouTube forces into SABR streaming

(drops to 144p).

97:To turn curated annotations into ground-truth files the badminton-highlight-indexer can score and train against, export per-video `start,end` CSVs plus a `manifest.json`:

100:python -m src.cli.curate export --out-dir data/eval_export

103:python -m src.cli.curate export --sport badminton --include-staging

105:The exporter writes `//.csv>` (decimal-second `start,end` rows) and a `manifest.json` pairing each CSV with its video `local\_path`/`video\_url`, fps, resolution, and license. The indexer keys videos by **file MD5**, not by our `video\_id`, so the downstream eval/training run must process the exact video file named in the manifest. Flags: `--sport` (restrict to one sport), `--include-staging` (default: curated only), `--include-noncommercial` (default: commercial-safe only).

121:- docs/PIPELINE_ROLE.md ? how this repo feeds the **badminton-highlight-indexer** (`export` / `fetch`).

26. user (2026-06-12T10:41:37.920Z)

```
usage: curate.py prep-annotation-proxy [-h] --video-path VIDEO_PATH
                                     [--out OUT] [--video-id VIDEO_ID]
                                     [--height HEIGHT] [--crf CRF]
                                     [--gop GOP] [--preset PRESET]
                                     [--manifest MANIFEST] [--allow-vfr]
                                     [--force]
```

options:

- h, --help show this help message and exit
- video-path VIDEO_PATH The ORIGINAL source video (stays the system-of-record file)
- out OUT Proxy output path (default: data/videos/proxies/<stem>_proxy.mp4)
- video-id VIDEO_ID Manifest key (default: derived from the filename, like import-local)
- height HEIGHT Max proxy height; never upscales (default: 1080 ? 720 risks losing the table-tennis ball)
- crf CRF x264 quality, lower=better (default: 19)
- gop GOP Max keyframe interval in frames; small = snappy frame-stepping (default: 15)
- preset PRESET x264 speed/size preset (default: medium)
- manifest MANIFEST Provenance CSV to append to (default: data/annotation_proxy_manifest.csv; commit it ? it is the proxy<->original mapping of record)
- allow-vfr Proceed on a variable-frame-rate source (frame/fps seconds are approximate; default: refuse)
- force Overwrite an existing proxy file

27. user (2026-06-12T10:42:28.374Z)

```
src/cli/curate.py:665: stem = os.path.basename(src).split('.')[0]
src/cli/curate.py:670: if os.path.abspath(out) == os.path.abspath(src):
src/cli/curate.py:702:     proxylib.run_ffmpeg(cmd)
src/cli/curate.py:727:     proxylib.append_manifest_row(args.manifest, {
src/core/proxy.py:210:     elif abs(proxy.avg_fps - src.avg_fps) / src.avg_fps > 0.001:
```

28. user (2026-06-12T10:43:31.824Z)

Structured output provided successfully

29. assistant (2026-06-12T10:43:40.202Z)

Review complete. Findings returned via structured output: one medium code issue (failed/partial proxy not deleted on encode/probe failure, violating the PR's own invariant), one medium doc gap (onboarding FAQ + diagram skip step 0), five low-severity items (Windows case-insensitive out==src guard, ZeroDivisionError edge via --allow-vfr, silent duplicate video_ids in the manifest, unhandled manifest-append failure, and the "identical by construction" claim vs the

actual 0.1%/0.2s verification tolerances), plus one ADR-wording drift note ? alongside an informational entry recording the lens items that verified clean (video_id parity incl. multi-dot, gitignore coverage, deps/typing/CI, downstream commands unaffected, 159 tests passing locally).